

Projeto: CineMatch

Recomendador de Filmes por Perfil

Visão Geral

Crie um sistema (incluindo diagrama de classes e diagrama de sequência) que recomenda filmes para uma pessoa com base no perfil dela: gêneros preferidos, duração ideal, classificação etária aceitável, idiomas, histórico do que já assistiu e notas que já deu para outros filmes. O sistema cruza o perfil com um catálogo de filmes e devolve uma lista ranqueada dos melhores candidatos.

Não há combate, turnos ou batalha. A mecânica central é calcular um score de compatibilidade entre um perfil e cada filme do catálogo, filtrar o que não faz sentido (filmes já assistidos, censurados para a faixa etária, em idioma não preferido) e ranquear o que sobra.

A dupla deve implementar a lógica, escrever testes unitários com JUnit 5 e usar Mockito para simular dependências externas — em especial o catálogo de filmes (que em produção viria de uma API tipo IMDB), o repositório de histórico do usuário (banco de dados) e a aleatoriedade usada em modos como “Surpreenda-me”.

Observação: “CineMatch” é o nome do projeto base. Para o pitch da turma, a dupla deve criar um nome próprio para o produto.

Como o Sistema Funciona — Exemplo de Uso Completo

Antes dos requisitos técnicos, leia este exemplo. Se você entender este exemplo, você entendeu o sistema todo.

Cenário

A Maria, 28 anos, abre o app pela primeira vez. Ela cria seu perfil e quer ver o que o sistema recomenda para ela hoje à noite.

Perfil da Maria (entrada)

```
Nome: Maria
Idade: 28
Preferências de gênero (peso de 0.0 a 1.0):
- Ficção Científica: 0.9
- Drama: 0.6
- Comédia: 0.5
- Terror: 0.0 (não gosta)
- Romance: 0.4
Duração preferida: 90 a 150 minutos
Classificação máxima: 16 anos
Idiomas aceitos: Português, Inglês
Já assistiu: ["Interestelar", "A Origem", "Matrix"]
Notas que já deu: {"Blade Runner 2049": 5, "Star Wars IX": 2}
```

Catálogo de filmes recebido (do CatalogoFilmesAPI)

ID	Título	Gêneros	Duração	Faixa	Idioma	Pop.
F01	Duna: Parte Dois	FC, Drama	166 min	14	EN	92
F02	Ela (Her)	FC, Drama, Romance	126 min	16	EN	78
F03	O Iluminado	Terror	146 min	18	EN	88
F04	Interestelar	FC, Drama	169 min	12	EN	95
F05	Tropa de Elite	Ação, Drama	115 min	18	PT	80
F06	Click	Comédia, Drama	107 min	12	EN	65
F07	A Chegada	FC, Drama	116 min	12	EN	84

Passo 1 — Filtragem (regras de eliminação)

O FiltroFilmes remove candidatos que violam regras absolutas do perfil:

- **F03 — O Iluminado:** gênero Terror tem peso 0.0 e classificação 18 > máxima 16 → eliminado.
- **F04 — Interestelar:** já está no histórico → eliminado.
- **F05 — Tropa de Elite:** classificação 18 > máxima 16 → eliminado.

Sobram: F01, F02, F06, F07.

Passo 2 — Cálculo de score (CalculadoraScore)

Para cada filme restante, o sistema calcula um score de 0 a 100. A fórmula leva em conta:

- **Compatibilidade de gênero** (50% do score) — média ponderada dos pesos dos gêneros do filme no perfil.
- **Aderência à duração** (20% do score) — 100 se está na faixa preferida; reduz proporcionalmente se está fora.
- **Popularidade do filme** (15% do score) — valor direto.
- **Bônus de afinidade** (15% do score) — se a pessoa deu nota alta a outros filmes do mesmo gênero, soma bônus.

Resultados:

Filme	Gênero	Duração	Pop.	Score final
F02 — Ela	$(0.9 + 0.6 + 0.4)/3 = 0.63$	100 (dentro)	78	78
F07 — A Chegada	$(0.9 + 0.6)/2 = 0.75$	100 (dentro)	84	82
F01 — Duna: Parte Dois	$(0.9 + 0.6)/2 = 0.75$	80 (16 min acima)	92	80
F06 — Click	$(0.5 + 0.6)/2 = 0.55$	100 (dentro)	65	62

Passo 3 — Ranqueamento

Ordena por score desc; em caso de empate, usa popularidade como desempate; se ainda empatar, usa o GeradorAleatorio (mockável).

Recomendação final para Maria:

1. A Chegada — score 82
2. Duna: Parte Dois — score 80
3. Ela (Her) — score 78
4. Click — score 62

O sistema também chama `HistoricoUsuario.registrarRecomendacao(...)` para guardar que recomendou esses filmes a ela hoje (evita repetição amanhã).

É isso. Recebe perfil + catálogo, filtra, pontua, ordena, devolve top N.

Conceitos do Sistema

Cada classe explicada em uma frase:

Conceito	Papel no sistema
Usuario	Pessoa que recebe recomendações. Tem nome, idade e um PerfilCinefilo.
PerfilCinefilo	Coleção de preferências: pesos por gênero, duração, classificação máxima, idiomas, histórico, notas.
Filme	Objeto de domínio. Tem título, ano, duração, gêneros, classificação, idioma, popularidade.
Genero	Enum: ACAO, COMEDIA, DRAMA, FICCAO_CIENTIFICA, ROMANCE, TERROR, DOCUMENTARIO etc.
Recomendacao	Resultado: filme + score calculado + justificativa textual.
CalculadoraScore	Calcula score 0–100 de um filme para um perfil. Lógica pura, NÃO mocka.
FiltroFilmes	Aplica regras de exclusão (já assistido, faixa etária, idioma). Lógica pura.
RecomendadorService	Orquestrador. Recebe perfil → busca catálogo → filtra → pontua → ordena → registra → devolve top N.
CatalogoFilmesAPI (mockável)	Devolve a lista de filmes disponíveis. Em produção: chamada HTTP para TMDB ou similar.
HistoricoUsuarioRepository (mockável)	Persiste e consulta histórico/notas do usuário. Em produção: banco de dados.
NotificadorPush (mockável)	Envia push avisando “saiu sua recomendação do dia”.
GeradorAleatorio (mockável)	Interface para sorteios (modo “Surpreenda-me”, desempate).

Requisitos do Projeto

1. Sistema de Perfil

- PerfilCinefilo permite cadastrar peso de gênero (0.0 a 1.0).
- Faixa de duração mínima e máxima.
- Classificação etária máxima aceitável (LIVRE, 10, 12, 14, 16, 18).
- Lista de idiomas aceitos.
- Histórico de filmes assistidos.
- Mapa de notas (ID do filme → nota de 1 a 5).
- Validações: peso fora de [0,1] lança exceção; duração mínima > máxima lança exceção; nota fora de [1,5] lança exceção.

2. Sistema de Catálogo

- Catálogo é fornecido pela interface CatalogoFilmesAPI (mockável).
- Mínimo de 30 filmes no catálogo de exemplo (pode ser hardcoded em uma classe CatalogoMock para uso fora dos testes).
- Filme tem mínimo 7 atributos: título, ano, duração, gêneros (lista), classificação, idioma, popularidade.

3. Filtragem

O FiltroFilmes deve aplicar pelo menos 4 regras:

- Excluir filmes já assistidos.
- Excluir filmes acima da classificação máxima.
- Excluir filmes em idioma não aceito.
- Excluir filmes com gênero de peso 0.0 (a pessoa explicitamente não quer).

4. Cálculo de Score

- Score é um número de 0 a 100.
- Composição: gênero (50%) + duração (20%) + popularidade (15%) + bônus por afinidade histórica (15%).
- Cada peso deve ser uma constante nomeada no código, não número mágico.

5. Recomendação

- Método principal: recomendar(Usuario u, int topN) → List<Recomendacao>.
- Retorna no máximo topN resultados.
- Lista vinda já ordenada por score desc, com desempate por popularidade desc, e desempate final aleatório (via GeradorAleatorio).
- Se o catálogo estiver vazio ou nada passar nos filtros: retornar lista vazia (nunca null).

6. Modo “Surpreenda-me”

- Método extra: recomendarAleatorio(Usuario u) → Recomendacao — pega 1 filme aleatório de quem passou no filtro.
- Usa GeradorAleatorio (mockável).

7. Persistência e Notificação

- Após cada recomendar(...), o sistema chama HistoricoUsuarioRepository.registrarRecomendacao(...) com a lista resultante.
- Se o usuário tiver notificações ligadas, chama NotificadorPush.enviar(...).
- Ambas as dependências DEVEM ser mockadas nos testes.

Estrutura de Classes Sugerida

```
src/
├── main/java/
│   ├── model/
│   │   ├── Usuario.java
│   │   ├── PerfilCinefilo.java
│   │   ├── Filme.java
│   │   ├── Recomendacao.java
│   │   └── enums/
│   │       ├── Genero.java
│   │       ├── ClassificacaoEtaria.java (LIVRE, DEZ, DOZE, QUATORZE, DEZESSEIS, DEZOITO)
│   │       └── Idioma.java
│   ├── service/
│   │   ├── RecomendadorService.java
│   │   ├── CalculadoraScore.java
│   │   ├── FiltroFilmes.java
│   │   ├── CatalogoFilmesAPI.java (interface)
│   │   ├── HistoricoUsuarioRepository.java (interface)
│   │   └── NotificadorPush.java (interface)
│   ├── exception/
│   │   ├── PesoInvalidoException.java
│   │   ├── DuracaoInvalidaException.java
│   │   └── PerfilIncompletoException.java
│   └── util/
│       └── GeradorAleatorio.java (interface)
└── test/java/
    ├── PerfilCinefiloTest.java
    ├── CalculadoraScoreTest.java
    ├── FiltroFilmesTest.java
    ├── RecomendadorServiceTest.java
    └── FilmeTest.java
```

Requisitos de Qualidade de Código

Avaliados na revisão. Código sem cuidado perde ponto mesmo que os testes passem.

5. **Nomenclatura clara** — Escolha português OU inglês e mantenha. Sem aux, temp1, x2, lista2.
6. **Responsabilidade única** — Filme não conhece API; RecomendadorService é o único orquestrador.
7. **Métodos curtos** — Idealmente até 20 linhas. Acima disso, extrair método privado com nome descritivo.
8. **Nunca retornar null para coleções** — Use Collections.emptyList(). Para objetos únicos opcionais, Optional<T>.
9. **Exceções específicas e nomeadas** — PesolInvalidoException etc. Nunca throw new RuntimeException("erro").
10. **Imutabilidade onde fizer sentido** — Filme, Recomendacao e os enums devem ser imutáveis.
11. **Sem números mágicos** — Todos os pesos da fórmula em static final.
12. **Injeção por construtor** — RecomendadorService recebe todas as dependências via construtor. Sem new de serviços externos dentro de métodos.
13. **Sem System.out.println** — Use uma interface de log dedicada se quiser logar.
14. **Javadoc** — Nos métodos públicos de RecomendadorService e nas interfaces externas.
15. **Cobertura de testes $\geq 80\%$** — Nos pacotes service/ e model/ (medido com JaCoCo).
16. **Sem código comentado** — Código morto vai para o histórico do Git.

Constantes da fórmula (exemplo):

```
static final double PESO_GENERO      = 0.50;
static final double PESO_DURACAO    = 0.20;
static final double PESO_POPULARIDADE = 0.15;
static final double PESO_AFINIDADE  = 0.15;
```


Requisitos de Testes Unitários

Cenários mínimos obrigatórios — pelo menos 20 métodos @Test

Sobre PerfilCinefilo

- Criar perfil com pesos válidos.
- Tentar criar perfil com peso < 0 ou > 1 lança `PesoInvalidoException`.
- Tentar criar perfil com duração mínima > máxima lança `DuracaoInvalidaException`.
- Adicionar nota fora de [1,5] lança exceção.
- Marcar filme como assistido aparece no histórico.

Sobre Filme

- Filme criado tem todos os atributos preenchidos.
- Dois filmes com mesmo ID são considerados iguais (equals/hashCode corretos).

Sobre CalculadoraScore

- Filme com todos os gêneros amados (peso 1.0) → score do componente gênero = 100.
- Filme com gênero não preferido → score baixo.
- Filme dentro da duração preferida → score do componente duração = 100.
- Filme com duração 30 min acima da preferência → score reduzido.
- Score nunca passa de 100 nem fica negativo (testes de limite).

Sobre FiltroFilmes

- Filme já assistido é removido.
- Filme acima da classificação é removido.
- Filme em idioma não aceito é removido.
- Filme com gênero de peso 0 é removido.
- Filtro com catálogo vazio devolve lista vazia (não null).

Sobre RecomendadorService (com Mockito)

- Recomendação devolve top N respeitando o tamanho pedido.
- Recomendação está ordenada por score desc.
- Empate de score é desempatado por popularidade.
- Catálogo vazio → recomendação vazia.
- API do catálogo lança exceção → serviço não derruba; trata e devolve vazio.
- Após recomendar, chama `historicoRepo.registrarRecomendacao(...)`.
- Se notificação está ligada, chama `notificador.enviar(...)`; se desligada, não chama.
- Modo “Surpreenda-me” devolve um filme do conjunto filtrado.

Cenário de integração (@Tag(“integracao”))

- Pipeline completo: catálogo mock realista + filtro real + calculadora real → ranking esperado.

Asserts que devem obrigatoriamente aparecer no código

Assertion	Onde encaixa neste sistema
<code>assertEquals(esperado, real)</code>	<code>assertEquals(82, score.getValor())</code> para A Chegada no exemplo.
<code>assertNotEquals(a, b)</code>	<code>assertNotEquals(rec1.getFilme(), rec2.getFilme())</code> — recomendações distintas.
<code>assertTrue(cond)</code>	<code>assertTrue(recomendacoes.get(0).getScore() >= recomendacoes.get(1).getScore())</code>
<code>assertFalse(cond)</code>	<code>assertFalse(recomendacoes.contains(filmeJaAssistido))</code>
<code>assertNull(obj)</code>	<code>assertNull(perfil.getNotaPara("filme-nunca-avaliado"))</code>
<code>assertNotNull(obj)</code>	<code>assertNotNull(recomendacao.getJustificativa())</code>
<code>assertThrows(Exc.class, () -> ...)</code>	<code>assertThrows(PesoInvalidoException.class, () -> perfil.setPeso(Genero.ACAO, 1.5))</code>
<code>assertArrayEquals(esp, real)</code>	Comparar IDs dos filmes na ordem esperada do ranking.
<code>assertAll(...)</code>	Validar todos os atributos de uma Recomendacao numa só chamada.
<code>assertDoesNotThrow(() -> ...)</code>	Garantir que falha de notificação não derruba a recomendação.

Anotações JUnit 5 obrigatórias

@BeforeEach — obrigatório em toda classe de teste com mais de 2 métodos

Por que: cada teste precisa de um perfil e um catálogo “limpos”. Sem @BeforeEach, mudanças no perfil de um teste vazariam para o seguinte.

```
@ExtendWith(MockitoExtension.class)
class RecomendadorServiceTest {

    @Mock private CatalogoFilmesAPI catalogo;
    @Mock private HistoricoUsuarioRepository historico;
    @Mock private NotificadorPush notificador;
    @Mock private GeradorAleatorio gerador;

    private CalculadoraScore calculadora;
    private FiltroFilmes filtro;
    private RecomendadorService service;
    private Usuario maria;

    @BeforeEach
    void setUp() {
        calculadora = new CalculadoraScore();           // lógica pura: instância real
        filtro      = new FiltroFilmes();               // lógica pura: instância real
        service     = new RecomendadorService(
            catalogo, historico, notificador, gerador, calculadora, filtro
        );
        maria = UsuarioFactory.maria();
    }
}
```

```
}
```

Outras anotações

- **@AfterEach** — usar quando houver recurso a liberar.
- **@DisplayName(“descrição legível”)** — obrigatório em todos os métodos de teste.
- **@Nested** — agrupar testes relacionados (ex.: “QuandoCatalogoEstaVazio”).
- **@ParameterizedTest com @CsvSource** — obrigatório em pelo menos 2 casos.
- **@Tag(“unitario”) / @Tag(“integracao”)** — separar tipos de teste.

Exemplos de @ParameterizedTest:

```
@ParameterizedTest
@CsvSource({
    "1.0, 1.0, 1.0, 100",
    "0.5, 0.5, 0.5, 50",
    "0.0, 0.0, 0.0, 0"
})
@DisplayName("score de gênero é a média ponderada dos pesos")
void deve_CalcularScoreDeGenero_ConformePesos(double a, double b, double c, int esperado) {
    ... }

@ParameterizedTest
@CsvSource({
    "DEZ, DOZE, false",
    "DEZESSEIS, DEZESSEIS, true",
    "DEZOITO, DEZESSEIS, false"
})
@DisplayName("filme é aceito quando classificação <= máxima do perfil")
void deve_AceitarFilme_QuandoClassificacaoEhCompativel(...) { ... }
```

Nomenclatura dos testes

Padrão obrigatório: `deve_ResultadoEsperado_Quando_Condicao()`

```
deve_LancarExcecao_Quando_PesoForaDoIntervalo()
deve_RemoverFilme_Quando_JaFoiAssistido()
deve_OrdenarPorScoreDesc_Quando_RecomendacaoTemMultiplosFilmes()
deve_RetornarListaVazia_Quando_CatalogoEstaVazio()
deve_ChamarNotificador_Quando_PushEstaHabilitado()
deve_NaoDerrubar_Quando_CatalogoLancaExcecao()
```

Estrutura AAA (Arrange–Act–Assert)

Todo teste tem três blocos visualmente separados:

```
@Test
@DisplayName("deve ordenar recomendações por score decrescente")
void deve_OrdenarPorScoreDesc_Quando_RecomendacaoTemMultiplosFilmes() {
    // Arrange
    List<Filme> catalogoMock = List.of(filmeBaixoScore, filmeAltoScore, filmeMedio);
    when(catalogo.buscarTodos()).thenReturn(catalogoMock);

    // Act
    List<Recomendacao> resultado = service.recomendar(maria, 3);

    // Assert
```

```
assertEquals(3, resultado.size());  
assertTrue(resultado.get(0).getScore() >= resultado.get(1).getScore());  
assertTrue(resultado.get(1).getScore() >= resultado.get(2).getScore());  
}
```

Requisitos de Uso de Mockito

O que DEVE ser mockado e por quê

Dependência	Por que mockar
CatalogoFilmesAPI	Em produção é uma chamada HTTP a uma API externa (TMDB/OMDB). Teste unitário não pode depender de internet.
HistoricoUsuarioRepository	Persistência (banco). Não queremos gravar dados a cada teste.
NotificadorPush	Serviço externo (Firebase/OneSignal). Mockar evita disparar push de verdade nos testes.
GeradorAleatorio	Sem mockar, o desempate aleatório e o modo “Surpreenda-me” deixariam os testes flaky (passam às vezes, falham às vezes).

O que NÃO faz sentido mockar

A dupla deve estar pronta para defender estas escolhas no debate:

- **CalculadoraScore** — lógica pura (matemática). Mockar esconderia bugs reais nas fórmulas.
- **FiltroFilmes** — lógica pura de filtragem. Igual.
- **Filme, Recomendacao, PerfilCinefilo** — objetos de domínio. Use instâncias reais.
- **Enums (Genero, ClassificacaoEtaria, Idioma)** — são valores; mockar enum não faz sentido.
- **A própria classe sob teste** — nunca se mocka o que está sendo testado.

Padrões de Mockito que devem aparecer no código

1. when(...).thenReturn(...) — stub básico

```
List<Filme> catalogoSimulado = List.of(filmeFC1, filmeFC2, filmeTerror);
when(catalogo.buscarTodos()).thenReturn(catalogoSimulado);

List<Recomendacao> resultado = service.recomendar(maria, 5);

assertEquals(2, resultado.size()); // terror foi filtrado
```

2. when(...).thenThrow(...) — simular falha de dependência

```
when(catalogo.buscarTodos()).thenThrow(new IOException("API offline"));

List<Recomendacao> resultado = service.recomendar(maria, 5);

assertTrue(resultado.isEmpty()); // serviço é resiliente
verify(notificador, never()).enviar(any());
```

3. Stub sequencial — múltiplos retornos consecutivos

```
when(gerador.sortearInteiro(0, 10)).thenReturn(2, 7, 0);
// usado em 3 chamadas seguidas no modo "Surpreenda-me" em sequência
```

4. verify(...) — confirmar chamadas no mock

```
service.recomendar(maria, 5);

verify(historico).registrarRecomendacao(eq(maria), anyList());
verify(historico, times(1)).registrarRecomendacao(any(), any());
verify(notificador, never()).enviar(any());           // notificações desligadas
verify(catalogo, atLeastOnce()).buscarTodos();
```

5. Matchers — any(), anyInt(), eq()

```
verify(historico).registrarRecomendacao(eq(maria), any(List.class));
```

Regra: se um argumento usa matcher, todos precisam usar matcher.

6. ArgumentCaptor — inspecionar o que foi passado ao mock

```
ArgumentCaptor<List<Recomendacao>> captor = ArgumentCaptor.forClass(List.class);

service.recomendar(maria, 3);

verify(historico).registrarRecomendacao(eq(maria), captor.capture());
List<Recomendacao> registradas = captor.getValue();

assertAll(
    () -> assertEquals(3, registradas.size()),
    () -> assertEquals("A Chegada", registradas.get(0).getFilme().getTitulo()),
    () -> assertTrue(registradas.get(0).getScore() >= registradas.get(2).getScore())
);
```

7. Anotações @Mock, @InjectMocks, @ExtendWith(MockitoExtension.class)

```
@ExtendWith(MockitoExtension.class)
class RecomendadorServiceTest {
    @Mock private CatalogoFilmesAPI catalogo;
    @Mock private HistoricoUsuarioRepository historico;
    @Mock private NotificadorPush notificador;
    @Mock private GeradorAleatorio gerador;
    @InjectMocks private RecomendadorService service;
}
```

8. Spy — usar em pelo menos 1 caso e justificar

Exemplo razoável: espionar uma instância real de CalculadoraScore para verificar quantas vezes o método de cálculo foi chamado durante uma recomendação, sem alterar o resultado.

Apresentação para o Professor — pelo menos 50% do projeto pronto - 1 ponto

A dupla deve estar pronta para tudo abaixo:

1. Visão geral e arquitetura

- Explicar a divisão model / service / exception / util.
- Mostrar o diagrama de classes.
- Mostrar o diagrama de sequência de uma recomendação: Usuario → RecomendadorService → CatalogoFilmesAPI → FiltroFilmes → CalculadoraScore → HistoricoUsuarioRepository → NotificadorPush.

2. Lista de cenários de teste

- Tabela: ID | Cenário | Entrada | Resultado esperado | Status. Mínimo 20 cenários.

3. Exemplos comentados de testes

- Pelo menos 3 testes JUnit puros explicando cada assertion (boas escolhas: testes da CalculadoraScore ou do FiltroFilmes).
- Pelo menos 3 testes com Mockito explicando cada when().thenReturn(), cada verify() e o porquê de cada mock.
- Pelo menos 1 uso de ArgumentCaptor justificado.

4. Bugs encontrados

Listar cada bug: descrição, qual teste revelou, correção aplicada. Se não acharam nenhum bug, os testes provavelmente estão fracos — revisem.

5. Reflexão crítica

- Quais dependências foram mockadas e por quê.
- O que NÃO faz sentido mockar (e por quê).
- Onde a dupla hesitou entre mock e objeto real, e como decidiu.

TESTE COM PERGUNTAS SOBRE O PROJETO - INDIVIDUAL - 1 PONTO

Apresentação para a Turma — 1 ponto (Pitch)

Cada dupla tem 5 a 10 minutos para apresentar como em um pitch de startup. Vendam o produto.

Estrutura obrigatória do pitch

1. Nome e identidade visual

- Nome próprio do produto (criem um — não usem “CineMatch”, esse é o nome do projeto base).
- Slogan curto (até 10 palavras). Ex.: “O filme certo, na noite certa, sem rolagem infinita.”
- Logo simples (Canva, Figma, ou desenhado e escaneado).

2. Problema que o produto resolve

- Quanto tempo as pessoas perdem escolhendo filme? Por que recomendações genéricas falham?
- Tragam pelo menos um dado real ou citação (ex.: pesquisa sobre “decision fatigue” em streamings).

3. Diferencial

- O que torna seu recomendador diferente do que Netflix/Prime já fazem?
- Possíveis ângulos: transparência da nota (mostra POR QUE recomendou), perfil ajustável de forma fina, modo “humor de hoje”, anti-bolha (recomenda fora da zona de conforto), etc.

4. Público-alvo

- Quem usaria? Cinéfilos? Famílias? Casais com gostos diferentes? Estudantes de cinema?

5. Demonstração de uso

- Mostrar uma recomendação real sendo gerada para um perfil de exemplo.
- Print/log do sistema funcionando, com a justificativa textual (“recomendamos A Chegada porque você gosta de FC + Drama e avaliou bem Blade Runner 2049”).

6. Demonstração técnica

- Rodar os testes ao vivo (JUnit + Mockito).
- Explicar um caso onde o Mockito foi decisivo — exemplo natural aqui: testar a recomendação sem depender da API de catálogo de verdade, e forçar um catálogo controlado para validar o ranking.

7. Considerações finais

- Principais desafios.
- O que aprenderam sobre testes que não sabiam antes.
- O que fariam diferente se comesçassem hoje.

Normas adicionais de apresentação

- **Tempo:** 5 a 10 min. Ultrapassar 11 min perde ponto.
- **Divisão:** ambos os integrantes devem falar. Apresentação solo não é permitida.

- **Slides:** máximo 12 slides, sem parágrafos longos. Bullets curtos.
- **Fonte e contraste:** mínimo 24pt; contraste alto (a sala pode ter reflexo).
- **Código na tela:** nunca em fonte menor que 18pt; recortar só o trecho relevante.
- **Demo ao vivo:** testes pré-executados e ambiente pronto. Tenham um GIF ou vídeo gravado como backup caso a demo trave.
- **Postura:** de pé, olhando para a turma, não para o slide.
- **Linguagem:** termo técnico novo deve ser explicado ao introduzi-lo.
- **Perguntas:** reservem pelo menos 2 min ao final para a turma.
- **Entrega junto do pitch:** repositório no GitHub com README contendo: como rodar, como testar, print da cobertura, diagrama de classes, diagrama de sequência, nomes dos integrantes.

Critérios de avaliação do pitch

Critério	Peso
Clareza e organização da apresentação	20%
Criatividade do conceito (nome, slogan, identidade visual)	15%
Qualidade da demonstração técnica ao vivo	25%
Explicação dos testes e do papel do Mockito	25%
Respeito ao tempo e postura da dupla	15%